WRO 2026 Future Engineers — Control & Tunables

Team The Red Castle — HMK AI and Robotics Club. Generated 2026-08-19 from the programs themselves; every value below is parsed out of the source at build time, so it cannot disagree with the code.

What the car actually measures

There is no distance sensor. Everything below is built from one 640×480 camera frame, cropped from row **CROP_TOP** to the bottom, taking every second column, then resized to **320×120**. All pixel counts, areas and thresholds in this document live on that 320×120 frame — which is why a threshold measured on a different crop does not transfer.

```
crop      = raw[CROP_TOP:480, 0:640:2]   ->  resize to 320x120

wall density  left_wall  = count(wall pixels, left half)  / 12800
              right_wall = count(wall pixels, right half) / 12800
```

The divisor is 12800 and each half is really 120×160 = 19200 pixels, so these densities are not true fractions — they run to about 1.5. That is inherited and harmless, because every target is measured on the same scale. It only matters if you compare against a number from elsewhere.

The wall mask — what counts as a wall

```
wall = (V < WALL_V_HARD)  OR  (V < WALL_V_SOFT AND S < WALL_S_MAX)

then  morphological OPEN, kernel WALL_OPEN_K

then  keep only pixels in a VERTICAL RUN of >= WALL_MIN_RUN pixels
```

Two cases, because saturation is unreliable when V is tiny — a black wall can report S>200 from sensor noise. Testing saturation alone rejects real walls; testing brightness alone accepts the coloured lines. The vertical-run test is the shadow filter: a real wall is a tall solid run, a shadow is a broad shallow smear.

Open Challenge — control law

One P controller on wall density. The car follows the OUTER wall: driving clockwise it turns right, so the inside of the loop is on its right and the LEFT wall is the outer one; counter-clockwise is the mirror.

```
direction = +1 (CW)    dir = (left_wall  - CW_TARGET ) * WALL_GAIN
direction = -1 (CCW)   dir = (CCW_TARGET - right_wall) * WALL_GAIN

before the direction locks (direction = 0):
   if left_wall  > NEUTRAL_TARGET:  dir = (left_wall - NEUTRAL_TARGET) * WALL_GAIN
   if right_wall > NEUTRAL_TARGET:  dir = (NEUTRAL_TARGET - right_wall) * WALL_GAIN

dir       = clamp(dir, -STEER_MAX, +STEER_MAX)
dir_servo = SERVO_SMOOTH * dir + (1 - SERVO_SMOOTH) * dir_servo_previous
pulse_us  = 500 + ((dir_servo + 90 + SERVO_TRIM) / 180) * 2000
```

TARGET is the density a CENTRED car reads. That is the whole calibration: park it in the middle, read the number, put it here. A target that is wrong by 0.03 commands about 2° of steering while the car is already centred — a constant lean into one wall, every cycle, all run.

Counting laps

```
pixels > threshold               -> state 1  (on the line)
state 1 -> below threshold       -> state 2  (crossed) -> quadrant + 1
then ignore that colour for LINE_BLANK_S seconds
12 quadrants -> keep driving FINISH_RUN_S -> stop
```

The count happens on the FALLING edge, so what causes a double count is the pixel count dipping below the threshold mid-crossing. A lower threshold makes that less likely, not more.

Open Challenge tunables

tunable	now	what it does	change it when
CW_TARGET	0.215	Left-wall density a centred car reads, driving clockwise.	Re-measure at the venue with **wall_calib.py cw**. Always.
CCW_TARGET	0.214	Right-wall density a centred car reads, counter-clockwise.	Re-measure with **wall_calib.py ccw**. Should be within ~0.01 of CW_TARGET.
NEUTRAL_TARGET	0.25	Before the direction locks, steer away from a wall past this density.	If the car drives straight out of the start into a wall, lower it.
WALL_GAIN	75	Degrees of steering per unit of density error.	Raise if it corrects too slowly; lower if it weaves on a straight.
STEER_MAX	20	Software steering limit for ordinary driving.	Raise toward 35 if it runs wide at corners. The log prints CLAMP.
SERVO_SMOOTH	0.35	Exponential average on the command. 1.0 = raw.	Lower if the steering is twitchy; raise if it feels sluggish.
SERVO_DEADBAND	0.8	Degrees of change below which the servo pulse is not rewritten.	Raise a little if the servo buzzes while going straight.
SERVO_TRIM	-9	Degrees added so that a command of 0 drives dead straight.	If the car curves with dir=0 in the log, adjust this, nothing else.
blue_line_threshould	830	Blue pixels needed to call it a line.	Set from **line_audit.py blue**: about 77% of a full line.
orange_line_threshould	930	Orange pixels needed to call it a line.	Set from **line_audit.py orange**.
LINE_BLANK_S	1.2	Seconds a colour is ignored after it is counted.	Must exceed one crossing (~0.5 s) and stay under the gap between counted lines (~5 s on a real run).
FINISH_RUN_S	3	Seconds of driving after the 12th quadrant.	Raise if the car stops on the line instead of past it.
CROP_TOP	160	First raw row kept. Lower = more wall in frame, smaller lines.	Changing this invalidates BOTH the wall targets and the line thresholds.
WALL_V_HARD	32	Below this brightness it is wall whatever the saturation says.	
WALL_V_SOFT	62	Up to this brightness it is wall only if desaturated.	

tunable	now	what it does	change it when
`WALL_S_MAX`	`90`	Saturation above this is a coloured line, not a wall.	
`WALL_MIN_RUN`	`6`	Vertical dark pixels required to be a wall — the shadow filter.	Raise if shadow leaks in; lower if distant walls vanish.
`speed`	`100`	Motor PWM percent.	Lower it before you tune anything else — every threshold is easier to hit slowly.

Obstacle Challenge — control law

Three things can steer, in this order of authority: a pillar, then a wall that is too close, then the corner kick. Between pillars there is no wall following at all — **dir is 0 and the car drives straight**.

Following a pillar

```
d = SIGN_Y_GAIN * (119 - y)          y is the blob centre; y=0 is FURTHEST

green aim = min(320 + GREEN_TARGET_CLAMP, GREEN_NEAR_{CW|CCW} + d)
red   aim = max(     -RED_TARGET_CLAMP,   RED_NEAR            - d)

Err(green) = -(green aim - x)        pass on its LEFT  -> steer LEFT
Err(red)   =  (x - red aim)          pass on its RIGHT -> steer RIGHT

if green and x > 220        : Err = 0      (release, CW only)
if red   and x <  90        : Err = 0      (release, CCW only)

dir = Err * kp,  clamped to GREEN_STEER_MAX or RED_STEER_MAX
```

The aim runs off-frame on purpose. The further away a pillar is, the further outside the 320-pixel frame the car aims, so it commits early and eases off as the pillar arrives. The clamps stop that becoming a lunge.

Measured, and still true in this build: the green release (`x > 220`, no distance condition) zeroes the steering on about 70% of the cycles a green pillar is held in CW, at any distance — down to x=221 with the pillar close. CCW has no such line, which is why green behaves better counter-clockwise. The distance-gated version is in git if it is wanted back.

Walls and the corner kick

```
if a wall density > WALL_CLOSE:  dir = +-(30 or 40) * that density
     ceiling becomes WALL_STEER_MAX

corner kick, once per crossing:
   CW  and last sign GREEN and orange line crossed -> servo(+45)
   CCW and last sign RED   and blue   line crossed -> servo(-45)

parking exit, once at the start:
   purple_left > purple_right -> exit RIGHT, direction = CW  (+1)
   otherwise                  -> exit LEFT,  direction = CCW (-1)
```

With no purple in view at all that comparison is false, so the car silently commits to CCW. Check the first line the program prints.

Obstacle Challenge tunables

tunable	now	what it does	change it when
kp	`0.05`	Degrees of steering per pixel of sign error.	The single strongest knob for pillar behaviour. Change it in small steps.
SIGN_Y_GAIN	`6.867`	How much further off-frame to aim per row of distance.	Raise to commit earlier to a far pillar; lower to react later.
GREEN_NEAR_CW	`300`	Where a CLOSE green pillar should sit in frame, driving CW.	Raise to pass wider around green. Try this before raising kp.
GREEN_NEAR_CCW	`262`	Same, counter-clockwise.	
RED_NEAR	`120`	Where a CLOSE red pillar should sit in frame.	Lower to pass wider around red.
GREEN_TARGET_CLAMP	`400`	How far past the frame edge green may be aimed. 400 = effectively free.	
RED_TARGET_CLAMP	`30`	Same for red. Small on purpose — it is what stops red lunging at the start of a section.	Raise if red is not avoided enough; lower if red over-reacts.
GREEN_STEER_MAX	`35`	Steering ceiling while following green.	Raise if green is clipped; the log shows the clamp.
RED_STEER_MAX	`15`	Steering ceiling while following red. Deliberately tighter than green.	Lower if red is still too sharp.

tunable	now	what it does	change it when
GREEN_MIN_AREA	50	Green blob size before it becomes a target. Area falls with the SQUARE of distance.	Keep low — green needs to start early to swing wide.
RED_MIN_AREA	400	Red blob size before it becomes a target.	Raise to answer red later. A pillar measures ~1220 px at mid distance.
PARALELIPIPED_MIN_AREA	75	Floor below which a blob is noise, not a pillar.	
SIGN_MAX_ASPECT	1	A blob is a pillar only if width < this × height.	Raise if a CLOSE pillar is lost — its base runs off frame so its height stops growing while its width does not.
SIGN_CLOSE_AREA_CW	500	Area at which the sign is latched as the last one seen (CW).	
SIGN_CLOSE_AREA_CCW	1005	Same, CCW.	
WALL_CLOSE	0.3	Density above which a wall overrides the pillar steering.	Lower if the car touches walls; raise if it shies away from them.
WALL_STEER_MAX	30	Ceiling for a wall correction, so a sign's tighter limit cannot blunt it.	
STEER_MAX	30	Ordinary steering limit when nothing special is happening.	
PARKING_ENABLED	False	End-of-run parking search. Currently OFF — the car finishes like the open challenge.	Set True to bring the whole parking algorithm back.
FINISH_RUN_S	3	Seconds of driving after the 12th quadrant, with parking off.	
blue_line_threshould	750	Blue pixels needed to call it a line.	From **line_audit.py**.
orange_line_threshould	840	Orange pixels needed. Orange shares its hue band with the RED pillars.	If phantom quadrants appear, raise this first.
LINE_BLANK_S	1.2	Seconds a colour is ignored after being counted.	
GREEN_SAT_MIN	150	Green saturation floor. The MAT sits inside the green hue band, so saturation is the ONLY thing separating cube from floor.	Measured gap: mat p99=146, cube p01=157. Do not go below ~150.
GREEN_VAL_MIN	25	Green brightness floor. The cube is DARK — V median 38.	At 60 it kept 1% of the cube. Leave low.
RED_SAT_MIN	120	Red saturation floor.	
PURPLE_SAT_MIN	120	Parking wall saturation floor.	
PURPLE_VAL_MIN	40	Parking wall brightness floor. The wall measures V p05=52 p50=67.	Raising this loses the wall; it is what the exit direction depends on.

Order of work at the venue

Do these in order. Each step assumes the ones above it are already right; doing them out of order means measuring through a mistake.

#	step	tool	why
1	Lock the camera	**mask_debug.py**	Look at the picture before any number. If the walls are not dark and the lines are not solid, nothing measured afterwards means anything. Venue lighting is the biggest single change between practice and competition.
2	Walls, both directions	**wall_calib.py cw** then **ccw**	Park CENTRED, read the density, put it in CW_TARGET / CCW_TARGET. The two should agree within about 0.01 — if they do not, the car was not centred for one of them.
3	Lines	**line_audit.py blue** / **orange**	Park over each line. Set the threshold near 77% of a full line. Check the frame-to-frame spread does not straddle it.
4	Pillars	**sign_calib.py**	Green and red at equal distance should give equal area and equal y. If one is much smaller, its colour range is clipping the cube — fix that before touching kp.
5	Parking walls	**park_calib.py**	Only for the obstacle start. Check both purple counts and that one side clearly wins — a near tie means a coin toss for the lap direction.
6	Drive, then read the log	logs/*.csv	Every run writes its own timestamped file. Do not tune from memory of what the car looked like; the log has the wall densities, the pixel counts, the target and the steering before and after the clamp.

Three rules that have cost this car whole runs

1. A threshold belongs to the crop it was measured on. CROP_TOP was raised from 240 to 160 to get the walls in frame, which squashes 320 rows into 120 instead of 240 into 120. Lines are horizontal, so they lose pixels in proportion: the same frame gives 1421 px of blue through the old crop and 1077 through this one. Both line thresholds were silently too high for months because of it.

2. Brightness floors set on a brighter camera reject the object. The same mistake three times: orange needed V>125 where the line reads 78; blue needed V>70 where it reads 43; green needed V>60 where the cube reads 38 and only 1% of it survived. If something is not detected, check the V floor before anything else.

3. A controller commanding zero looks exactly like a controller that is happy. The neutral wall target was 0.5 on a camera whose densities reach 0.25, so before the direction locked the car steered nothing at all and drove straight out of the start into the wall. Read dir in the log, not the car.